Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Asimptotik notasyonlar: Big O, Big Omega, Big Theta
  - Bir algoritmanın büyüme mertebesi (order of growth) genelde $1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < \cdots < 2^n < 3^n < \cdots < n^n$ sıralamasındaki sınıflardan biridir ya da bunların birleşimidir (ekrandan, 00:20:00).
  - Big O üst sınırı verir: $f(n) \in O(g(n))$, ancak ve ancak pozitif bir sabit c ve negatif olmayan bir tam sayı n_0 için her $n \geq n_0$ değerinde $f(n) \leq c\,g(n)$ ise.
  - Big Omega alt sınırı verir: her $n \geq n_0$ için $f(n) \geq c\,g(n)$.
  - Big Theta iki taraftan sınırlar: her $n \geq n_0$ için $c_1 g(n) \leq f(n) \leq c_2 g(n)$. Hoca bunu "en kullanışlı" notasyon olarak tanıttı.
  - Big O örneği (ekrandan, 00:10:00): $f(n) \leq 5n$ $(n \geq 1)$ yazılabildiği için $f(n) \in O(n)$; $f(n) \leq 7n^2$ yazılabildiği için $f(n) \in O(n^2)$ de doğrudur, ama daha az kullanışlıdır.
  - Big Theta örneği (ekrandan, 00:16:50): $\frac{1}{2}n(n-1) \in \Theta(n^2)$. Üst sınır için $c_2 = \frac{1}{2}$ $(n \geq 0)$; alt sınır için $c = 1, 2, \frac{1}{2}$ denenir, tutmaz; $c_1 = \frac{1}{4}$ tutar. $n_0 = 2$ seçilir.
  - Diğer örnekler (ekrandan, 00:19:20 ve 00:20:00): $n^2 \leq 4n^2 + 5n + 4 \leq 9n^2$, yani $\Theta(n^2)$; $n^2 \log n \leq n^2 \log n + n \leq 5n^2 \log n$, yani $\Theta(n^2 \log n)$.
- Ardışık iki parçadan oluşan algoritmalar
  - $t_1(n) \in O(g_1(n))$ ve $t_2(n) \in O(g_2(n))$ ise $t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$. Genel karmaşıklığı büyük olan parça belirler.
  - İspatta kullanılan gözlem: $a_1 \leq b_1$ ve $a_2 \leq b_2$ ise $a_1 + a_2 \leq 2\max\{b_1, b_2\}$. $c_3 = \max\{c_1, c_2\}$ ve $n_0 = \max\{n_1, n_2\}$ alınır.
  - Örnek: önce sıralayıp sonra aynı elemandan var mı diye taramak. Sıralama $\frac{1}{2}n(n-1)$ karşılaştırmayla $O(n^2)$, tarama $n-1$ ile $O(n)$; toplam $O(n^2)$.
- Büyüme mertebelerini limitle karşılaştırma (ekrandan, 00:30:00)
  - $\lim_{n\to\infty} t(n)/g(n)$ sıfırsa t'nin büyümesi daha küçüktür, sabit c ise aynıdır, sonsuzsa daha büyüktür.
  - $\frac{1}{2}n(n-1)$ ile n^2: limit $\frac{1}{2}$, aynı mertebe, $\Theta(n^2)$.
  - $n!$ ile 2^n (ekrandan, 00:33:10): Stirling formülü $n! \approx \sqrt{2\pi n}\,(n/e)^n$ ile limit $\sqrt{2\pi n}\,(n/2e)^n \to \infty$. $n!$ çok daha hızlı büyür.
- Özyinelemeli olmayan algoritmaların analizi
  - MaxElement (ekrandan, 00:37:20): temel işlem karşılaştırma, $C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$.
  - Genel plan: girdi boyutu parametresini seç, temel işlemi belirle, temel işlemin sayısı yalnız girdi boyutuna mı bağlı diye bak (gerekirse en kötü, ortalama, en iyi durumu ayrı incele), toplam formülünü kur ve sadeleştir.
  - UniqueElements (ekrandan, 00:40:00 ve 00:43:20): iç içe iki döngü, $C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2)$.
  - MatrixMultiplication (ekrandan, 00:45:20): üç iç içe döngü, $M(n) = \sum\sum\sum 1 = n^3$.
  - Binary (ekrandan, 01:04:40): n'i her adımda ikiye bölerek ikilik basamak sayısını sayar; döngü $\lfloor \log_2 n \rfloor + 1$ kez döner, $\Theta(\log n)$.
- Özyinelemeli algoritmaların analizi
  - Faktöriyel: $F(n) = F(n-1) \cdot n$. Çarpım sayısı için $M(n) = M(n-1) + 1$, $M(0) = 0$.

Backward substitution ile $M(n) = M(n-i) + i$, $i = n$ konursa $M(n) = n \in \Theta(n)$ (ekrandan, 01:10:00).
  - Genel plan: girdi boyutu parametresi, temel işlem, durum analizi, recurrence relation ve başlangıç koşulu, recurrence'ı çözme.
  - Hanoi Kuleleri (ekrandan, 01:18:50): $M(n) = 2M(n-1) + 1$, $M(1) = 1$. Çözüm $M(n) = 2^n - 1 \in \Theta(2^n)$.
  - Özyinelemeli çağrı ağacı: her çağrı bir düğümdür. Hanoi'de düğüm sayısı toplam hamle sayısını verir.
  - BinRec (ekrandan, 01:20:00): $A(n) = A(\lfloor n/2 \rfloor) + 1$, $A(1) = 0$. Smoothness rule ile $n = 2^k$ alınır, $A(2^k) = k$, yani $A(n) = \log_2 n \in \Theta(\log n)$ (ekrandan, 01:29:20).
  - Fibonacci (ekrandan, 01:30:00): $F(n) = F(n-1) + F(n-2)$, $F(0) = 0$, $F(1) = 1$. Çağrı ağacının dalları farklı derinlikte biter. Karmaşıklık $\Theta(\phi^n)$, $\phi \approx 1{,}618$ (altın oran); 2^n yalnız bir üst sınırdır.
- Alıştırmalar
  - Büyüme mertebesi karşılaştırmaları (ekrandan, 01:36:40), toplam ifadesinin mertebesi (ekrandan, 01:38:50), Mystery, en büyük ile en küçük farkını bulan algoritma, Enigma ve kare ızgara problemi. Çözümler "Detaylı Açıklamalar" bölümündedir.

Hocanın Özellikle Vurguladığı Kısımlar

- Notasyonda en yakın sınırı seçmek
  - $2n + 3$ için $O(n)$, $O(n^2)$, hatta $O(n^3)$ teknik olarak doğrudur. Ama sınırdan uzaklaştıkça bilgi değeri düşer; en kullanışlı ifade en yakın olanıdır.
- Tanımdaki sabitlerin önemi yok, varlıkları önemli
  - Bir öğrencinin sorusu üzerine hoca, $5n^2 \log n$'deki 5'in özel olmadığını, eşitsizliği sağlayan bir (c, n_0) çiftinin var olmasının yettiğini söyledi.
- Analizde n sonsuza giderken olan önemlidir
  - 100 ya da 200 elemanlı dizide fark yoktur; fark binlerde, milyonlarda ortaya çıkar.
- Limit yöntemi notasyondan daha bilgilendiricidir
  - $n! \in \Omega(2^n)$ doğrudur ama Big Omega "eşit ya da büyük" dediği için eşit büyüme olasılığını eleyemez. Limit sonsuz çıkınca $n!$'in kesin olarak daha hızlı büyüdüğü görülür.
- Recurrence relation'da başlangıç koşulu
  - Çağrının nerede bittiğini (initial condition) doğru belirlemek gerekir; analiz bunun üzerine kurulur. Faktöriyelde $M(0) = 0$, Hanoi'de $M(1) = 1$, BinRec'te $A(1) = 0$.
- Pratik yapmak
  - Hoca konunun oturması için kendi örneklerini çözmeyi ve toplam formüllerini kullanmayı önerdi.

Kısa Tekrar Notları

- O: üst sınır, Ω: alt sınır, Θ: iki yönlü sıkı sınır.
- Sabit katsayılar ve düşük dereceli terimler büyüme mertebesini değiştirmez.
- Ardışık parçalar: $O(\max\{g_1, g_2\})$.
- Limit: 0 küçük, c aynı, ∞ büyük.
- $n! \approx \sqrt{2\pi n}\,(n/e)^n$ (Stirling).
- $\sum_{i=l}^{u} 1 = u - l + 1$, $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$, $\sum_{i=1}^{n} i^k \approx \frac{1}{k+1} n^{k+1}$.
- MaxElement $\Theta(n)$; UniqueElements $\Theta(n^2)$; matris çarpımı $\Theta(n^3)$; Binary $\lfloor \log_2 n \rfloor + 1$.
- Faktöriyel $M(n) = n$; Hanoi $2^n - 1$; BinRec $\log_2 n$; Fibonacci $\Theta(\phi^n)$.
- $\log_2 n$ ile $\ln n$ aynı mertebededir: $\log_2 n = \frac{1}{\ln 2} \ln n$.
- $n!$, $(n-1)!$'den daha yüksek mertebededir: $n! = n\,(n-1)!$ ve çarpan n'e bağlıdır.
- $\sum_{i=0}^{n-1} (i^2 + 1)^2 \in \Theta(n^5)$.
- Kare ızgara problemi: $C(n) = C(n-1) + 4n$, $C(0) = 1$, $C(n) = 2n^2 + 2n + 1 \in \Theta(n^2)$.

Detaylı Açıklamalar

- **Tanımlar ve örnekler.** f, g gerçel sayılardan gerçel sayılara tanımlı fonksiyonlardır. $f(n) \in O(g(n))$ demek için pozitif bir c ve negatif olmayan bir n_0 bulunmalı ve her $n \geq n_0$ için $f(n) \leq c\,g(n)$ olmalıdır. Slayttaki örnek $2n+3$ için $7n$, $2n+3n = 5n$ ve $4n^2+3n^2 = 7n^2$ yazılabilir (ekrandan, 00:10:00). Eşitsizliğin sağında tek bir terim olmalı ve eşitsizlik sağlanmalıdır. $f(n) \in O(\log n)$ denemez; Big O, bulunulan noktanın solundaki küçük seçenekleri eler. Big Omega'da aynı fonksiyon için $1 \cdot n$ ve $\log n$ alt sınırdır, yani $f(n) \in \Omega(n)$ ve $\Omega(\log n)$; ama $\Omega(n^2)$ yazılamaz. Big Theta'da $2n+3$ için $c_1 = 1$, $c_2 = 5$ ile $\Theta(n)$ elde edilir.
- **Ders sonrası not (slayt yazımı).** Slaytta örnek fonksiyon "2n*3" diye yazılmış, hoca da ilk anlatımda "2n çarpı 3" diye okudu (ekrandan, 00:10:00). Hoca Big Theta kısmında aynı fonksiyonu "2n artı 3" diye andı (00:14:13). $2n \cdot 3 = 6n$ hiçbir $n \geq 1$ için $5n$'den küçük olmaz; slayttaki "$\leq 5n$" sınırı ancak $2n+3$ için doğrudur. Fonksiyon büyük olasılıkla $2n+3$'tür.
- **Ders sonrası not (Big Theta'nın adı).** Hoca Big Theta için "average bound" dedi. Big Theta ortalama durum (average case) anlamına gelmez; alttan ve üstten aynı mertebeyle sınırlayan sıkı sınırdır (tight bound). Hoca da notasyonların geçen haftaki en kötü/en iyi/ortalama durum kavramıyla aynı şey olmadığını belirtti.
- $\frac{1}{2}n(n-1) \in \Theta(n^2)$ **ispatı** (ekrandan, 00:16:50). Üst sınır: $\frac{1}{2}n^2 - \frac{n}{2} \leq \frac{1}{2}n^2$, $n \geq 0$ için doğrudur; $c_2 = \frac{1}{2}$. Alt sınır: $\frac{1}{2}n^2 - \frac{1}{2}n \geq c_1 n^2$ için $c_1 = 1$, 2 ve $\frac{1}{2}$ tutmaz, $\frac{1}{4}$ tutar. n_0 için $n = 1$ denenir, tutmaz ($0 \geq \frac{1}{4}$ yanlış); $n = 2$ ve sonrası tutar. Seçilen $n_0 = 2$ (python ile doğrulandı).
- $4n^2 + 5n + 4$ **ve** $n^2 \log n + n$**.** Slaytta $4n^2 + 5n + 4 \geq n^2$ ($\Omega(n^2)$) ve $4n^2 + 5n + 4 \leq 9n^2$ ($O(n^2)$), dolayısıyla $\Theta(n^2)$ (ekrandan, 00:19:20). Hoca n_0'ın makul seçilmesi gerektiğini söyledi: üst eşitsizlik $n \geq 2$ için tutar. $n^2 \log n + n$ fonksiyonu n^2 ile n^3 arasındadır.
- **Ders sonrası not (grafikteki sınırlar).** Hoca Big Theta grafiğini anlatırken üstteki eğriyi "Big O üst sınırım", alttakini "Big Theta, yani alt sınırım" dedi (00:21:31). Alttaki eğri $c_1 g(n)$, yani Big Omega tipinde alt sınırdır; Big Theta iki eğrinin arasındaki bölgedir.
- **Sıralama + tarama örneği.** Hoca anlatırken parçaların rollerini karıştırır gibi oldu: sıralamayı "tüm diziyi tarıyorsanız $n-1$" diye, $\frac{1}{2}n(n-1)$'lik algoritmayı ise "işi bitiren" diye andı. Slayttaki örnek büyük olasılıkla şöyledir: sıralama $\frac{1}{2}n(n-1)$ karşılaştırma ($O(n^2)$), sıralı dizide yan yana elemanları tarama $n-1$ karşılaştırma ($O(n)$); toplam $O(n^2)$. Bellek açısından hoca, uygulamaya bağlı olmakla birlikte normalde n'lik bir alanın yeteceğini söyledi.
- **Limit örnekleri.** $\lim \frac{\frac{1}{2}n(n-1)}{n^2} = \frac{1}{2}\lim(1-\frac{1}{n}) = \frac{1}{2}$; ters bölmek sonucu değiştirmez (sabit yine pozitif bir sabittir). $n!/2^n$ için Stirling ile $\sqrt{2\pi n}\,\frac{n^n}{2^n e^n} = \sqrt{2\pi n}\,(\frac{n}{2e})^n \to \infty$.
- **UniqueElements türetmesi** (ekrandan, 00:43:20). $C_{worst}(n) = \sum_{i=0}^{n-2}[(n-1)-(i+1)+1] = \sum_{i=0}^{n-2}(n-1-i) = (n-1)\sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2} = (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2}$. En iyi durumda ilk karşılaştırmada çıkılır.
- **Binary izlemesi.** $n = 5$ için: $count = 1$; $5 > 1$, $count = 2$, $n = 2$; $2 > 1$, $count = 3$, $n = 1$; döngü biter, sonuç 3 ($5 = 101_2$). Sondaki $+1$, döngüden çıkmadan önceki son kontrolden gelir.
- **Hanoi backward substitution** (ekrandan, 01:18:50). $M(n) = 2[2M(n-2)+1]+1 = 2^2M(n-2)+2+1 = 2^3M(n-3)+2^2+2+1$. Genel: $M(n) = 2^iM(n-i)+2^i-1$. $n-i=1$ için $i = n-1$: $M(n) = 2^{n-1}M(1)+2^{n-1}-1 = 2^n-1$.
- **Smoothness rule.** $n/2$ aşağı yuvarlandığı için backward substitution ikinin kuvveti olmayan n'lerde zorlaşır. Kural, $n = 2^k$ için bulunan büyüme mertebesinin yuvarlamadan etkilenmediğini söyler. $A(2^k) = A(2^{k-1})+1 = \cdots = A(2^{k-k})+k = k$, $k = \log_2 n$.

- **Fibonacci.** Doğrudan tanımdan yazılan özyinelemeli algoritma optimum değildir ama analiz için iyi bir örnektir. Backward substitution burada karışır; recurrence'ı çözmek için homojen olmayan recurrence'ı homojene çevirme teknikleri gerekir ve sonuçta altın oran ortaya çıkar. Ağaç her seviyede tam dolu olsaydı düğüm sayısı 2^n mertebesinde olurdu; dallar farklı derinlikte bittiği için gerçek sayı bunun altındadır.
- **Ders sonrası not (Fibonacci sınırı).** Hoca "Big Omega'da 2^n evet, ama Big Theta'da 2^n değil" dedi (01:33:42). Çağrı sayısı $\Theta(\phi^n)$ ve $\phi < 2$ olduğu için doğru ifade $O(2^n)$'dir; $\Omega(2^n)$ değildir. Hoca hemen ardından "2^n Big O'da doğru" diyerek bunu düzeltti.
- **Alıştırma çözümleri.** $n(n+1)$ ile $2000n^2$ aynı mertebe; $100n^2$ karesel, $0{,}01n^3$ kübik; $\log_2 n = \frac{\ln n}{\ln 2}$ olduğundan aynı mertebe; $n! = n\,(n-1)!$ olduğundan $n!$ daha yüksek mertebe (ekrandan, 01:36:40). $\sum_{i=0}^{n-1}(i^2+1)^2 = \sum(i^4+2i^2+1) \approx \sum_{i=1}^{n} i^4 - n^4 \approx \frac{1}{5}n^5 - n^4 \in \Theta(n^5)$ (ekrandan, 01:38:50).
- **Mystery** (ekrandan, 01:40:00): 1'den n'e kadar sayıların karelerini toplar. Döngü n kez döner, $\Theta(n)$. İyileştirme: kareler toplamının kapalı formülüyle $\Theta(1)$.
- **En büyük ve en küçük farkı** (ekrandan, 01:47:00): dizideki en büyük ve en küçük elemanı bulup farklarını döndürür. Döngü içinde iki karşılaştırma vardır: $C(n) = \sum_{i=1}^{n-1} 2 = 2(n-1) \in \Theta(n)$. İkinci `if` yerine `else if` kullanmak bazı karşılaştırmaları atlatır ama mertebeyi değiştirmez.
- **Enigma** (ekrandan, 01:50:00): matrisin simetrik olup olmadığını kontrol eder. Temel işlem $A[i,j]$ ile $A[j,i]$ karşılaştırmasıdır. $\sum_{i=0}^{n-2}(n-1-i) = (n-1)+(n-2)+\cdots+1 = \frac{(n-1)n}{2} \in \Theta(n^2)$. Transpozunu alıp karşılaştırmak yaklaşık iki kat iş yapar; o da $\Theta(n^2)$'dir.
- **Ders sonrası not (Enigma slaytı).** Slaytta koşul `if A[i, j] = A[j, i] return false` diye yazılı (ekrandan, 01:50:00). Hocanın anlattığı simetri kontrolü için koşul büyük olasılıkla $A[i,j] \neq A[j,i]$ olmalıdır; eşitlikle yazılırsa simetrik matriste hemen `false` döner.
- **Kare ızgara** (ekrandan, 02:00:00): 0. iterasyonda ortada 1 kare boyalıdır; her iterasyonda boyalı alanın dört komşuluğundaki kareler eklenir. Eklenen kare sayısı 4, 8, 12, yani $4n$'dir. $C(0) = 1$, $C(1) = 5$, $C(2) = 13$, $C(3) = 25$. Recurrence $C(n) = C(n-1) + 4n$. Backward substitution ile $C(n) = C(0) + 4\sum_{i=1}^{n} i = 1 + 4\frac{n(n+1)}{2} = 2n^2 + 2n + 1 \in \Theta(n^2)$ (python ile doğrulandı).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.